The University of Lahore
Department of Computer Science & IT
Mid Term Examination Solution

Spring-2024

Course Title:	Object Oriented Programming	Course Code:	CS02203	Credit Hours:	4
Course Instructors:	Dr.Mulahammad Atif, Mr. Ali Raza, Ms. Iqra Adnan, Ms Haseena Kainat, Mr. Sardar Waqar Khan	Program Name:	BSCS		
Time Allowed:	**60 Minutes**	**Maximum**	**25**		
Date:	**18-03-2024**	Course Mentor:	Dr. Muhammad Atif Chattha		
Student's		Signature:			
Reg. No:		Section:			

Note:

1) It is an open book exam.

2) Understanding questions is part of examination, so no queries will be answered.

.

Question # 1: **(10 Marks)**

Design a class called "Laptop" to model laptop devices. Each instance of the "Laptop" class should possess attributes representing its brand, model, and owner's name. Implement a copy constructor for the "Laptop" class, which initializes a new laptop object with the same brand and model as the original object, but with an empty owner's name. Additionally, provide a setter method named "setOwnerName" to allow the assignment of an owner's name to a laptop object after its creation.

In the main function, demonstrate the utilization of the "Laptop" class by performing the following steps:

1. Create an original instance of the "Laptop" class.
2. Copy the original laptop object to a new laptop object.
3. Utilize the "setOwnerName" method to assign an owner's name to the new laptop object.
4. Display the information of each laptop, including brand, model, and owner's name.
5. Calculate and display the average price of the laptops.

You have the freedom to determine whether additional constructors or methods are necessary to achieve the desired functionality..

Solution:-

```
#include <iostream>

#include <string>

using namespace std;

class Laptop {

private:
```

```
    string brand;
    string model;
    string owner_name;
    double price;
public:
    // Constructor
    Laptop(string brand, string model, double price, string owner_name){
        this->brand = brand;
        this->model = model;
        this->owner_name = owner_name;
        this->price = price;
        }
    // Copy constructor
    Laptop(const Laptop &other){
        brand = other.brand;
        model = other.model;
        price = other.price;
        owner_name = "";
        }
    // Setter method for owner's name
    void setOwnerName(string owner_name) {
        this->owner_name = owner_name;
    }
    // Display laptop information
    void displayInfo() {
        cout << "Brand: " << brand << ", Model: " << model << ", Price: "<< price <<", Owner's
Name: " << owner_name << endl;
    }
    // Calculate Average Price
```

```
    int getprice(){
        return price;
    }
    /*double CalculateAvg(Laptop obj){
        double avg = (price+ obj.price)/2;
        return avg;
    }*/
};
int main() {
    // Create an original instance of the Laptop class
    Laptop original_laptop("Dell", "Inspiron 15", 10000, "Doe");
    // Copy the original laptop object to a new laptop object
    Laptop new_laptop = original_laptop;
    // Utilize the "setOwnerName" method to assign an owner's name to the new laptop object
    new_laptop.setOwnerName("John Doe");


    // Display the information of each laptop
    cout << "Original Laptop: ";
    original_laptop.displayInfo();
    cout << "New Laptop: ";
    new_laptop.displayInfo();
    double average = (original_laptop.getprice() + new_laptop.getprice()) / 2;
    cout<<average;
    /cout<< original_laptop.CalculateAvg(new_laptop);/
    return 0;
}
```

Question # 2: (15 Marks)

Design a class "Rectangle" with attributes length and width. Create a derived class "PlayArea" from "Rectangle" and add an attribute "surfaceType". Additionally, create another derived class "PlayGround" from "PlayArea" with an attribute for the number of players. In the main function, instantiate three objects: football ground, lawn tennis ground, and basketball ground. Display the data of the playground with the maximum area. Also, calculate and display the average number of players across all grounds. You have the discretion to decide whether additional constructors or methods are required for functionality where no data member is public.

Solution:-

```
#include <iostream>
#include <string>
using namespace std;
class Rectangle {
protected:
   double length;
   double width;
public:
   // Constructor
   Rectangle(){
      length = 0;
      width = 0;
      }
   // Copy Constructor
   Rectangle(double l, double w){
      length = l;
      width = w;
      }
      //Calculate Area
      double calculateArea() {
         return length * width;
```

```
    }
};
//Derived Class
class PlayArea : public Rectangle {
protected:
   string surfaceType;
public:
//Constructor
   PlayArea(double l, double w, string surface): Rectangle(l , w){
   surfaceType = surface;
   }
   //Return value of Surface
   string getSurfaceType() {
      return surfaceType;
   }
};
//Sub Drive class
class PlayGround : public PlayArea {
private:
   int numOfPlayers;
public:
   //Constructor
   PlayGround(double l, double w, string surface, int players) : PlayArea(l, w, surface) {
      numOfPlayers = players;
   }
   //Return value of Players
   int getNumOfPlayers() {
      return numOfPlayers;
```

```
    }
    //Display function
    void DisplayData(){
        cout<<"Length: "<<length<<endl;
        cout<<"Width: "<<width<<endl;
        cout<<"Surface: "<<surfaceType<<endl;
        cout<<"Number of Player: "<<numOfPlayers;
    }
};
int main() {
    PlayGround footballGround(70, 36, "Grass", 22);
    PlayGround tennisGround(100, 50, "Clay", 4);
    PlayGround basketballGround(94, 50, "Hardwood", 10);
    // Find the playground with maximum area
    double maxArea = footballGround.calculateArea();
    double tennisArea = tennisGround.calculateArea();
    double basketballArea = basketballGround.calculateArea();
    if (tennisArea > maxArea) {
        if(tennisArea>basketballArea){
            tennisGround.DisplayData();
        }
    }
    else if (basketballArea > maxArea) {
        basketballGround.DisplayData();
    }
    else{
        footballGround.DisplayData();
    }
```

```
    // Calculate average number of players
    double averagePlayers;
    averagePlayers = (footballGround.getNumOfPlayers() +
                      tennisGround.getNumOfPlayers() +
                      basketballGround.getNumOfPlayers()) / 3.0;
    cout << "\nAverage number of players across all grounds: " << averagePlayers << endl;


    return 0;
}
```

Good Luck